RELEVANT SOURCE CODE

Function: generate_qr_code_with_crypto (QR Code with Encryption)

```
def generate_qr_code_with_crypto(certificate_data):
    """
    Generate a QR code with encrypted certificate data.
    Args:
        certificate_data: Dictionary with certificate data including
ZKP proof

    Returns:
        ContentFile with the QR code image
    """
    print("Starting QR code generation for certificate ID:",
certificate_data.get('cert_id', 'unknown'))

    try:
        loggable_data = certificate_data.copy()
        if 'verification_code' in loggable_data:
            loggable_data['verification_code'] = '*****'
        print("QR generation input data:", loggable_data)

        if 'zkp' in certificate_data:
            print("ZKP proof detected in certificate data")
            if certificate_data['zkp']:
                print("ZKP proof length:",
len(str(certificate_data['zkp'])), "chars")
            else:
                print("Empty ZKP proof found in certificate data")

        encrypted_data = encrypt_qr_data(certificate_data)
        print("Encrypted data length:", len(encrypted_data), "chars")

        qr = qrcode.QRCode(
            version=None,
            error_correction=qrcode.constants.ERROR_CORRECT_M,
            box_size=4,
            border=2,
        )
        qr.add_data(encrypted_data)
        qr.make(fit=True)

        img = qr.make_image(fill_color="black", back_color="white")

        buffer = BytesIO()
        img.save(buffer, format="PNG")
        buffer.seek(0)

        print("Successfully generated QR code for certificate ID:",
certificate_data.get('cert_id', 'unknown'))
        return ContentFile(buffer.getvalue())

    except Exception as e:
```

```
        print("Error generating QR code:", str(e))
        raise
```

Function: encrypt_qr_data (Encrypt Certificate Data for QR Code)

```
def encrypt_qr_data(data_dict):
    """
    Encrypt certificate data to make it unreadable by standard QR
scanners.

    Args:
        data_dict: Dictionary with certificate data

    Returns:
        String with encrypted data ready for QR code
    """
    logger.debug("Starting QR data encryption")

    try:
        if 'zkp' in data_dict:
            logger.debug("Encrypting data containing ZKP proof")
            zkp_size = len(json.dumps(data_dict['zkp'])) if
data_dict['zkp'] else 0
            logger.debug(f"ZKP proof size before encryption: {zkp_size}
bytes")

        json_string = json.dumps(data_dict)
        logger.debug(f"Original JSON size: {len(json_string)} bytes")

        data_bytes = json_string.encode('utf-8')
        base64_encoded = base64.b64encode(data_bytes).decode('utf-8')
        logger.debug(f"Base64 encoded size: {len(base64_encoded)}
chars")

        prefixed_data = f"ECAS-CERT:{base64_encoded}"

        key = "ECASCryptoKey2024"
        xor_result = []

        for i, char in enumerate(prefixed_data):
            key_char = key[i % len(key)]
            xor_char = chr(ord(char) ^ ord(key_char))
            xor_result.append(xor_char)

        obfuscated_data = ''.join(xor_result)
        logger.debug(f"Obfuscated data size: {len(obfuscated_data)}
chars")

        final_data = base64.b64encode(obfuscated_data.encode('utf-
8')).decode('utf-8')
        logger.debug(f"Final encrypted data size: {len(final_data)}
chars")

        return final_data

    except Exception as e:
        logger.error(f"Error encrypting QR data: {str(e)}")
```

```
        logger.error(traceback.format_exc())
        raise
```

Function: decrypt_qr_data (Decrypt Certificate Data from QR Code)

```
def decrypt_qr_data(encoded_content):
    """
    Decrypt the content from a secure certificate QR code.

    Args:
        encoded_content: The raw string content from the QR code scan

    Returns:
        dict: Decoded certificate data or None if invalid
    """
    try:
        print("\n" + "="*80)
        print("QR CODE PAYLOAD DECRYPTION - STARTED")
        print("="*80)
        print(f"[DEBUG] Encoded content length: {len(encoded_content)}
chars")

        try:
            obfuscated_data =
base64.b64decode(encoded_content).decode('utf-8')
            print(f"[DEBUG] Base64 decode successful, obfuscated data
length: {len(obfuscated_data)} chars")
        except Exception as e:
            print(f"[ERROR] DECRYPTION BLOCKED: Base64 decoding
failed")
            print(f"  - Error: {str(e)}")
            print("  - Possible cause: QR payload corrupted or
modified")
            print("="*80 + "\n")
            logger.warning(f"Base64 decoding failed: {str(e)}")
            return None

        key = "ECASCryptoKey2024"
        deobfuscated_chars = []

        for i, char in enumerate(obfuscated_data):
            key_char = key[i % len(key)]
            original_char = chr(ord(char) ^ ord(key_char))
            deobfuscated_chars.append(original_char)

        deobfuscated_content = ''.join(deobfuscated_chars)
        print(f"[DEBUG] XOR deobfuscation complete, content preview:
{deobfuscated_content[:50]}...")

        prefix = "ECAS-CERT:"
        if not deobfuscated_content.startswith(prefix):
            print(f"\n[ERROR] DECRYPTION BLOCKED: Invalid QR payload -
missing expected prefix")
            print(f"  - Expected prefix: '{prefix}'")
            print(f"  - Found: '{deobfuscated_content[:20]}...'")
            print("  - Possible cause: QR code payload has been
tampered with or is not a valid certificate QR")
```

```
            print("="*80 + "\n")
            logger.warning(f"Missing prefix, found:
{deobfuscated_content[:15]}...")
            return None

        print(f"[DEBUG] Prefix validation passed: '{prefix}'")
        encoded_json = deobfuscated_content[len(prefix):]

        try:
            json_data = base64.b64decode(encoded_json).decode('utf-8')
            print(f"[DEBUG] JSON data decoded, length: {len(json_data)}
chars")
        except Exception as e:
            print(f"\n[ERROR] DECRYPTION BLOCKED: Base64 decoding of
JSON data failed")
            print(f"  - Error: {str(e)}")
            print("  - Possible cause: QR payload structure corrupted")
            print("="*80 + "\n")
            logger.warning(f"Base64 decoding of JSON data failed:
{str(e)}")
            return None

        try:
            decoded_data = json.loads(json_data)

            print(f"\n[DEBUG] QR Payload Successfully Decrypted:")
            print(f"  - Keys found: {list(decoded_data.keys())}")
            print(f"  - Data preview: {str(decoded_data)[:200]}...")

            if 'zkp' in decoded_data:
                zkp_length = len(str(decoded_data['zkp']))
                print(f"  - ZKP proof: FOUND (length: {zkp_length}
chars)")
                logger.info(f"ZKP proof found in QR code data, length:
{zkp_length} chars")
            else:
                print(f"  - ZKP proof: MISSING - VERIFICATION WILL
FAIL!")
                print("\n[WARNING] No ZKP proof in QR code payload")
                print("  - This QR code cannot be verified securely")
                logger.warning("No ZKP proof in QR code data")

            print("="*80 + "\n")
            logger.info(f"Decrypted data keys:
{list(decoded_data.keys())}")
            logger.info(f"Decrypted data preview:
{str(decoded_data)[:200]}")

            return decoded_data

        except json.JSONDecodeError as e:
            print(f"\n[ERROR] DECRYPTION BLOCKED: JSON parsing failed")
            print(f"  - Error: {str(e)}")
            print("  - Possible cause: QR payload data structure
corrupted or modified")
            print("="*80 + "\n")
            logger.warning(f"JSON parsing failed: {str(e)}")
```

```
            return None

    except Exception as e:
        print(f"\n[ERROR] DECRYPTION BLOCKED: Unexpected error during
QR decoding")
        print(f"  - Error: {str(e)}")
        print(f"  - Traceback: {traceback.format_exc()}")
        print("="*80 + "\n")
        logger.error(f"Error decoding QR content: {str(e)}")
        logger.error(traceback.format_exc())
        return None
```

Function: process_qr_content (Multi-Method QR Data Extraction)

```
def process_qr_content(raw_qr_data):
    """
    Process QR content to extract certificate data including ZKP proof.

    Args:
        raw_qr_data: String from QR code scan

    Returns:
        dict: Decoded certificate data or None if invalid
    """
    logger.info("Processing QR content")

    diagnostics = {
        "attempted_methods": [],
        "raw_length": len(raw_qr_data),
        "raw_preview": raw_qr_data[:30] + "..." if len(raw_qr_data) >
30 else raw_qr_data
    }

    try:
        diagnostics["attempted_methods"].append("direct_json")
        json_data = json.loads(raw_qr_data)
        logger.info("QR content is valid JSON")

        if 'unique_id' in json_data or 'verification_code' in
json_data:
            logger.info("Found certificate identifiers in JSON")

            if 'zkp' in json_data:
                logger.info("Found ZKP proof in direct JSON data")
            else:
                logger.warning("No ZKP proof in direct JSON data")

            diagnostics["success_method"] = "direct_json"
            return json_data
        else:
            logger.info("JSON doesn't contain certificate identifiers")
    except json.JSONDecodeError:
        logger.info("QR content is not valid JSON")

    try:
        diagnostics["attempted_methods"].append("main_decryption")
```

```
        decrypted_data = decrypt_qr_data(raw_qr_data)
        if decrypted_data:
            logger.info("Successfully decrypted with main method")
            logger.info(f"Decrypted keys:
{list(decrypted_data.keys())}")

            if 'zkp' in decrypted_data:
                logger.info(f"Found ZKP proof in decrypted data,
length: {len(str(decrypted_data['zkp']))}")
            else:
                logger.warning("No ZKP proof in decrypted data")

            diagnostics["success_method"] = "main_decryption"
            return decrypted_data
    except Exception as e:
        logger.info(f"Main decryption failed: {str(e)}")

    try:
        diagnostics["attempted_methods"].append("simple_decoding")
        simple_decoded = decode_simple_encrypted_qr(raw_qr_data)
        if simple_decoded:

            if 'zkp' in simple_decoded:
                logger.info("Found ZKP proof in simple decoded data")
            else:
                logger.warning("No ZKP proof in simple decoded data")

            diagnostics["success_method"] = "simple_decoding"
            return simple_decoded
    except Exception as e:
        logger.info(f"Simple decoding failed: {str(e)}")

    try:
        diagnostics["attempted_methods"].append("base64_json")
        base64_decoded = base64.b64decode(raw_qr_data).decode('utf-8')
        json_data = json.loads(base64_decoded)
        logger.info("Successfully decoded as base64 JSON")

        if 'zkp' in json_data:
            logger.info("Found ZKP proof in base64 JSON data")
        else:
            logger.warning("No ZKP proof in base64 JSON data")

        diagnostics["success_method"] = "base64_json"
        return json_data
    except Exception:
        logger.info("Not base64 JSON")

    try:
        diagnostics["attempted_methods"].append("double_base64")
        decoded_once = base64.b64decode(raw_qr_data).decode('utf-8')
        decoded_twice = base64.b64decode(decoded_once).decode('utf-8')
        json_data = json.loads(decoded_twice)
        logger.info("Successfully decoded as double base64 JSON")

        if 'zkp' in json_data:
            logger.info("Found ZKP proof in double base64 JSON data")
```

```
            else:
                logger.warning("No ZKP proof in double base64 JSON data")

            diagnostics["success_method"] = "double_base64"
            return json_data
        except Exception:
            logger.info("Not double base64 JSON")

        try:
            diagnostics["attempted_methods"].append("special_format")
            if raw_qr_data.startswith("cert:"):
                logger.info("Found 'cert:' prefix")
                cert_id = raw_qr_data[5:]
                logger.warning("Using cert: prefix format, which doesn't
contain ZKP proof")
                return {"verification_code": cert_id}
        except Exception:
            pass

        logger.error("Failed to process QR content with any method")
        logger.debug(f"QR processing diagnostics: {diagnostics}")
        return None
```

Function: generate_unique_id (Secure Unique ID for Certificates)

```
import hashlib
import uuid
import base64
import json
import qrcode
import logging
import os
import traceback
from io import BytesIO
from django.core.files.base import ContentFile
from ecdsa import SigningKey, NIST256p, VerifyingKey
from ecdsa.util import sigencode_der, sigdecode_der
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives.serialization import Encoding,
PublicFormat, PrivateFormat, NoEncryption

logger = logging.getLogger(__name__)

def generate_unique_id():
    """Generate a unique secure identifier for the certificate."""
    raw_id = str(uuid.uuid4())
    unique_id = hashlib.sha256(raw_id.encode()).hexdigest()
    return unique_id
```

Function: generate_ecc_keys (Generate ECC Private and Public Keys)

```
def generate_ecc_keys():
    """Generate an ECC key pair for digital signatures."""
    try:
        sk = SigningKey.generate(curve=NIST256p)
```

```
        vk = sk.verifying_key

        private_key_pem = sk.to_pem().decode('utf-8')
        public_key_pem = vk.to_pem().decode('utf-8')

        private_key_hash =
hashlib.sha256(private_key_pem.encode()).hexdigest()

        return {
            'private_key': private_key_pem,
            'public_key': public_key_pem,
            'private_key_hash': private_key_hash
        }
    except Exception as e:
        logger.error(f"Error generating ECC keys: {str(e)}")
        logger.error(traceback.format_exc())

        private_key = ec.generate_private_key(ec.SECP256R1())
        public_key = private_key.public_key()

        private_key_pem = private_key.private_bytes(
            encoding=Encoding.PEM,
            format=PrivateFormat.PKCS8,
            encryption_algorithm=NoEncryption()
        ).decode('utf-8')

        public_key_pem = public_key.public_bytes(
            encoding=Encoding.PEM,
            format=PublicFormat.SubjectPublicKeyInfo
        ).decode('utf-8')

        private_key_hash =
hashlib.sha256(private_key_pem.encode()).hexdigest()

        return {
            'private_key': private_key_pem,
            'public_key': public_key_pem,
            'private_key_hash': private_key_hash
        }
```

Function: generate_deterministic_ecc_keys (Seed-Based ECC Key Pair Generation)

```
def generate_deterministic_ecc_keys(seed_string):
    """
    Generate deterministic ECC keys from a seed string.
    Same seed always produces the same keys.
    Args:
        seed_string: String to use as seed (e.g., verification_code)

    Returns:
        dict: Keys dictionary with private_key, public_key,
private_key_hash
    """
    try:
        seed_hash = hashlib.sha256(seed_string.encode()).digest()
```

```
        seed_int = int.from_bytes(seed_hash, byteorder='big')

        sk = SigningKey.from_secret_exponent(seed_int % NIST256p.order,
curve=NIST256p)
        vk = sk.verifying_key

        private_key_pem = sk.to_pem().decode('utf-8')
        public_key_pem = vk.to_pem().decode('utf-8')

        private_key_hash =
hashlib.sha256(private_key_pem.encode()).hexdigest()

        logger.info(f"Generated deterministic keys from seed:
{seed_string[:10]}...")

        return {
            'private_key': private_key_pem,
            'public_key': public_key_pem,
            'private_key_hash': private_key_hash
        }
    except Exception as e:
        logger.error(f"Error generating deterministic ECC keys:
{str(e)}")
        logger.error(traceback.format_exc())
        return generate_ecc_keys()
```

Function: create_certificate_data (Prepare Certificate Data for Signature)

```
def create_certificate_data(certificate):
    """Create a standardized representation of certificate data for
signing."""
    cert_data = {
        'cert_id': str(certificate.id),
        'student_id': certificate.user.student_id,
        'student_name': f"{certificate.user.first_name}
{certificate.user.last_name}",
        'cert_type': certificate.cert_type,
        'issuing_authority': certificate.issuing_authority,
        'issue_date': str(certificate.issue_date),
        'unique_id': certificate.unique_id,
        'timestamp': str(certificate.created_at)
    }
    return json.dumps(cert_data, sort_keys=True)
```

Function: generate_digital_signature (Create ECC-Based Signature)

```
def generate_digital_signature(private_key_pem, data):
    """Create a digital signature using the ECC private key."""
    try:
        sk = SigningKey.from_pem(private_key_pem)
        signature = sk.sign(data.encode(), sigencode=sigencode_der)
        return base64.b64encode(signature).decode('utf-8')
    except Exception as e:
        logger.error(f"Error generating digital signature: {str(e)}")
        logger.error(traceback.format_exc())
```

```
        return None
```

Function: verify_digital_signature (Validate ECC-Based Signature)

```
def verify_digital_signature(public_key_pem, data, signature):
    """Verify a digital signature using the public key."""
    try:
        vk = VerifyingKey.from_pem(public_key_pem)
        signature_bytes = base64.b64decode(signature)
        return vk.verify(signature_bytes, data.encode(),
sigdecode=sigdecode_der)
    except:
        return False
```

Function: generate_zkp_proof_for_qr (Create Compact ZKP Proof)

```
def generate_zkp_proof_for_qr(certificate_data, private_key_pem):
    """
    Generate a Zero-Knowledge Proof for the QR code.
    This function creates a compact ZKP proof suitable for QR code
inclusion.

    Args:
        certificate_data: String JSON of certificate data
        private_key_pem: Private key in PEM format

    Returns:
        str: JSON string with compact ZKP proof components
    """
    try:
        from ecdsa import SigningKey, NIST256p
        import hashlib
        import base64
        import os
        import json

        sk = SigningKey.from_pem(private_key_pem)
        vk = sk.verifying_key

        cert_hash =
hashlib.sha256(certificate_data.encode()).hexdigest()

        k = int.from_bytes(os.urandom(32), byteorder='big') %
NIST256p.order

        R = k * NIST256p.generator

        R_bytes = R.to_bytes()

        pub_bytes = vk.to_string()

        challenge_input = R_bytes + pub_bytes + cert_hash.encode()
        e = int.from_bytes(
            hashlib.sha256(challenge_input).digest(),
            byteorder='big'
        ) % NIST256p.order
```

```
        x = sk.privkey.secret_multiplier
        s = (k - e * x) % NIST256p.order

        compact_zkp = {
            'R': base64.b64encode(R_bytes).decode('utf-8'),
            'e': format(e, 'x'),
            's': format(s, 'x')
        }

        return json.dumps(compact_zkp)

    except Exception as e:
        import logging
        import traceback
        logger = logging.getLogger(__name__)
        logger.error(f"Error generating ZKP for QR: {str(e)}")
        logger.error(traceback.format_exc())
        return None
```

Function: encrypt_certificate_data (Prototype ECC-Based Encryption)

```
def encrypt_certificate_data(certificate_data, public_key_pem):
    """
    Encrypt certificate data using ECC public key.
    This is a placeholder for full encryption implementation.
    """
    encrypted_data = hashlib.sha256((certificate_data +
public_key_pem).encode()).hexdigest()
    return encrypted_data
```

Method: init (Initialize Certificate Verification with ZKP

```
def __init__(self, certificate, qr_extracted_zkp=None):
        """
        Initialize with a certificate object from the database and ZKP
from QR

        Args:
            certificate: Certificate model instance
            qr_extracted_zkp: ZKP proof extracted from QR code (JSON
string)
        """
        self.certificate = certificate
        self.qr_zkp = qr_extracted_zkp
        self.status = {
            'zkp_verified': False,
            'signature_verified': False,
            'otp_verified': False,
            'overall_verified': False,
            'error': None
        }
```

Method: verify_all (Validate ZKP and Digital Signature)

```
def verify_all(self):
        """

        Returns:
            dict: Status dictionary with verification results
        """
        try:
            certificate_data = self._create_certificate_data()

            if self.qr_zkp:
                zkp_result = self.verify_qr_zkp_proof(certificate_data)
                self.status['zkp_verified'] = zkp_result

                if not zkp_result:
                    self.status['error'] = "ZKP verification failed"
                    return self.status
            else:
                logger.warning("No ZKP proof from QR code - skipping
ZKP verification")
                self.status['zkp_verified'] = True

            sig_result =
self.verify_digital_signature(certificate_data)
            self.status['signature_verified'] = sig_result

            if not sig_result:
                self.status['error'] = "Digital signature verification
failed"
                return self.status


            self.status['overall_verified'] = True

            return self.status

        except Exception as e:
            logger.error(f"Error in certificate verification:
{str(e)}")
            logger.error(traceback.format_exc())
            self.status['error'] = str(e)
            return self.status
```

Method: _create_certificate_data (Generate JSON for Verification)

```
 def _create_certificate_data(self):
        """

        Returns:
            str: JSON string of certificate data
        """
        cert = self.certificate
        data = {
            'cert_id': str(cert.id),
```

```
            'student_id': cert.user.student_id,
            'student_name': f"{cert.user.first_name}
{cert.user.last_name}",
            'cert_type': cert.cert_type,
            'issuing_authority': cert.issuing_authority,
            'issue_date': str(cert.issue_date) if cert.issue_date else
None,
            'unique_id': cert.unique_id,
            'timestamp': str(cert.created_at)
        }

        return json.dumps(data, sort_keys=True)
```

Method: verify_qr_zkp_proof (Validate Schnorr ZKP from QR)

```
def verify_qr_zkp_proof(self, certificate_data):
        """
        Verify the ZKP proof extracted from QR code

        Args:
            certificate_data: JSON string of certificate data

        Returns:
            bool: True if verification successful
        """
        try:
            print("\n" + "="*80)
            print("CHECKING QR ZKP PROOF INTEGRITY")
            print("="*80)

            if not self.qr_zkp:
                print("[ERROR] VERIFICATION BLOCKED: No ZKP proof
provided from QR code")
                print("="*80 + "\n")
                logger.error("No ZKP proof provided from QR code")
                return False

            try:
                print(f"\n[DEBUG] Raw QR ZKP proof (first 100 chars):
{str(self.qr_zkp)[:100]}...")
                proof_data = json.loads(self.qr_zkp)
                print(f"[DEBUG] ZKP proof keys found:
{list(proof_data.keys())}")

                if all(k in proof_data for k in ['R', 'e', 's']):
                    print("[DEBUG] Detected Schnorr ZKP proof format")
                    logger.info("Detected Schnorr ZKP proof, using
elliptic curve verification")
                    return self._verify_schnorr_qr_zkp(proof_data,
certificate_data)

                print(f"\n[ERROR] VERIFICATION BLOCKED: Unknown ZKP
proof format")
                print(f"  - Expected keys: R, e, s")
                print(f"  - Found keys: {',
'.join(proof_data.keys())}")
```

```
                print("="*80 + "\n")
                logger.error("Unknown ZKP proof format with keys: " +
", ".join(proof_data.keys()))
                return False

            except json.JSONDecodeError as e:
                print(f"\n[ERROR] VERIFICATION BLOCKED: Invalid ZKP
proof format - not valid JSON")
                print(f"  - Error: {str(e)}")
                print("="*80 + "\n")
                logger.error(f"Invalid ZKP proof format - not valid
JSON: {str(e)}")
                return False

        except Exception as e:
            print(f"\n[ERROR] VERIFICATION BLOCKED: ZKP verification
error")
            print(f"  - Error: {str(e)}")
            print(f"  - Traceback: {traceback.format_exc()}")
            print("="*80 + "\n")
            logger.error(f"ZKP verification error: {str(e)}")
            logger.error(traceback.format_exc())
            return False
```

Method: _verify_schnorr_qr_zkp (Validate Commitment and Response in ZKP Proof)

```
def _verify_schnorr_qr_zkp(self, proof_data, certificate_data):
        """
        Verify a Schnorr protocol ZKP from QR code

        Args:
            proof_data: The parsed proof data
            certificate_data: Certificate data string

        Returns:
            bool: True if verification successful
        """
        try:
            print("\n" + "="*80)
            print("QR ZKP VERIFICATION DEBUG - STARTED")
            print("="*80)

            R_b64 = proof_data.get('R')
            e_hex = proof_data.get('e')
            s_hex = proof_data.get('s')

            print(f"\n[DEBUG] QR ZKP Proof Components:")
            print(f"  - R (base64): {R_b64[:50]}..." if R_b64 and
len(R_b64) > 50 else f"  - R (base64): {R_b64}")
            print(f"  - e (hex): {e_hex}")
            print(f"  - s (hex): {s_hex}")

            if not (R_b64 and e_hex and s_hex):
                print("\n[ERROR] VERIFICATION BLOCKED: Invalid Schnorr
ZKP format - missing components")
```

```
                print("="*80)
                logger.error("Invalid Schnorr ZKP format - missing
components")
                return False

            try:
                R_bytes = base64.b64decode(R_b64)
                e = int(e_hex, 16)
                s = int(s_hex, 16)
                print(f"\n[DEBUG] Decoded ZKP Components:")
                print(f"  - R bytes length: {len(R_bytes)} bytes")
                print(f"  - e integer: {e}")
                print(f"  - s integer: {s}")
            except Exception as decode_err:
                print(f"\n[ERROR] VERIFICATION BLOCKED: Error decoding
Schnorr ZKP components")
                print(f"  - Error: {str(decode_err)}")
                print("="*80)
                logger.error(f"Error decoding Schnorr ZKP components:
{str(decode_err)}")
                return False

            public_key_pem = self.certificate.public_key
            try:
                vk = VerifyingKey.from_pem(public_key_pem)
                pubkey_point = vk.pubkey.point
                pubkey_bytes = vk.to_string()
                print(f"\n[DEBUG] Certificate Public Key:")
                print(f"  - Public key point X: {pubkey_point.x()}")
                print(f"  - Public key point Y: {pubkey_point.y()}")
            except Exception as key_err:
                print(f"\n[ERROR] VERIFICATION BLOCKED: Failed to parse
public key")
                print(f"  - Error: {str(key_err)}")
                print("="*80)
                logger.error(f"Failed to parse public key:
{str(key_err)}")
                return False

            cert_hash =
hashlib.sha256(certificate_data.encode()).hexdigest()
            print(f"\n[DEBUG] Certificate Data Hash: {cert_hash}")


            g = NIST256p.generator
            curve = NIST256p

            gs_point = g * s

            ye_point = pubkey_point * e


            computed_R = gs_point + ye_point

            try:
                if len(R_bytes) == 64:
```

```
                    x_int = int.from_bytes(R_bytes[:32],
byteorder='big')
                    y_int = int.from_bytes(R_bytes[32:],
byteorder='big')
                    R_point_from_bytes = NIST256p.curve.point(x_int,
y_int)
                else:
                    R_point_from_bytes =
VerifyingKey.from_string(R_bytes, curve=curve).pubkey.point
            except Exception:
                R_point_from_bytes = None

            if R_point_from_bytes:
                result = (computed_R.x() == R_point_from_bytes.x() and
                          computed_R.y() == R_point_from_bytes.y())
            else:
                computed_R_ser = computed_R.to_bytes()
                result = computed_R_ser == R_bytes

            if result:
                print("\n[SUCCESS] QR ZKP VERIFICATION PASSED")
                print(f"  - The computed R point matches the commitment
in the QR ZKP proof")
                print("="*80 + "\n")
                logger.info("Schnorr ZKP verification from QR
successful")
            else:
                print("\n[CRITICAL ERROR] QR ZKP VERIFICATION FAILED!")
                print("="*80)
                print("VERIFICATION BLOCKED: The QR ZKP proof does NOT
match!")
                print("="*80)
                print("\nREASON: The computed R point does NOT match
the commitment")
                print("\nPOSSIBLE CAUSES:")
                print("  1. QR code payload has been tampered with or
modified")
                print("  2. QR code was generated with different data
than certificate")
                print("  3. ZKP proof in QR code is corrupted or
invalid")
                print("  4. Certificate data has been altered after QR
generation")
                print("\nACTION: Verification process STOPPED -
certificate is NOT verified")
                print("="*80 + "\n")
                logger.error("Schnorr ZKP verification from QR failed -
computed point doesn't match commitment")
                logger.error(f"QR ZKP mismatch details - R from proof:
{R_b64[:50]}..., Computed R coords: ({computed_R.x()},
{computed_R.y()})")

            return result

        except Exception as e:
            logger.error(f"Schnorr ZKP verification error: {str(e)}")
            logger.error(traceback.format_exc())
```

```
            return False
```

Method: verify_digital_signature (Validate ECC-Based Signature on Certificate Data)

```
def verify_digital_signature(self, certificate_data):
        """
        Verify the digital signature of the certificate

        Args:
            certificate_data: JSON string of certificate data

        Returns:
            bool: True if signature is valid
        """
        try:
            signature_b64 = self.certificate.digital_signature
            public_key_pem = self.certificate.public_key

            if not signature_b64 or not public_key_pem:
                logger.error("Missing signature or public key")
                return False

            try:
                signature_bytes = base64.b64decode(signature_b64)
            except Exception as e:
                logger.error(f"Error decoding signature: {str(e)}")
                return False

            try:
                vk = VerifyingKey.from_pem(public_key_pem)
                result = vk.verify(signature_bytes,
certificate_data.encode(), sigdecode=sigdecode_der)
                logger.info(f"Digital signature verification result:
{result}")
                return result
            except Exception as e:
                logger.warning(f"Digital signature verification error:
{str(e)}")
                return False

        except Exception as e:
            logger.error(f"Digital signature verification error:
{str(e)}")
            return False
```

Method: generate_otp (Create Random Numeric One-Time Password)

```
def generate_otp(length=6):
    """Generate a random numeric OTP"""
    return ''.join(random.choice(string.digits) for _ in range(length))
```

Method: store_otp (Save One-Time Password in Cache with Expiration)

```
def store_otp(email, certificate_id, otp):
```

```
    """Store OTP in cache with expiration time"""
    cache_key = f"cert_otp_{email}_{certificate_id}"
    cache.set(cache_key, otp, 300)
    return cache_key
```

Method: verify_otp (Validate Provided OTP Against Stored Value)

```
def verify_otp(email, certificate_id, provided_otp):
    """Verify the provided OTP"""
    cache_key = f"cert_otp_{email}_{certificate_id}"
    stored_otp = cache.get(cache_key)

    if not stored_otp:
        return False, "Verification code has expired. Please request a
new one."

    if stored_otp == provided_otp:
        cache.delete(cache_key)
        return True, "Verification successful"

    return False, "Invalid verification code. Please try again."
```

Method: send_otp_email (Send One-Time Password to User via Email)

```
def send_otp_email(email, otp, name, certificate_info):
    """Send OTP to user's email"""
    subject = "Certificate Verification Code"
    message = f"""
Hello {name},

Your verification code for certificate {certificate_info} is:

{otp}

This code will expire in 5 minutes.

Thank you,
Certificate Verification System
"""

    try:
        send_mail(
            subject,
            message,
            settings.DEFAULT_FROM_EMAIL,
            [email],
            fail_silently=False,
        )
        return True, "Verification code sent successfully"
    except Exception as e:
        logger.error(f"Error sending email: {str(e)}")
        return False, f"Error sending verification code: {str(e)}"
```

Method: generate_otp (Generate Numeric One-Time Password of Specified Length)

```
def generate_otp(length=6):
    """Generate a numeric OTP of specified length"""
    return ''.join(random.choice(string.digits) for _ in range(length))
```

Method: store_otp (Store One-Time Password in Cache with 5-Minute Expiration)

```
def store_otp(email, certificate_id, otp):
    """Store OTP in cache with an expiration time (5 minutes)"""
    cache_key = f"certificate_otp_{email}_{certificate_id}"
    cache.set(cache_key, otp, 300)
    return cache_key
```

Method: verify_otp (Check Provided OTP Against Stored OTP in Cache)

```
def verify_otp(email, certificate_id, provided_otp):
    """Verify if the provided OTP matches the stored OTP"""
    cache_key = f"certificate_otp_{email}_{certificate_id}"
    stored_otp = cache.get(cache_key)

    if not stored_otp:
        return False, "OTP has expired or is invalid. Please request a
new one."

    if stored_otp == provided_otp:
        cache.delete(cache_key)
        return True, "OTP verified successfully"

    return False, "Invalid OTP. Please try again."
```

Method: send_otp_to_verifier (Send One-Time Password to Verifier via Email)

```
def send_otp_to_verifier(email, otp, verifier_name, student_name,
cert_type):
    """Send OTP to the verifier's email"""
    subject = "Certificate Verification OTP"
    message = f"""
Hello {verifier_name},

You are about to verify a {cert_type} certificate for {student_name}.

To complete the verification process, please use the following One-Time
Password (OTP):

{otp}

This OTP is valid for 5 minutes. If you did not request this
verification, please contact your administrator.

Regards,
Educational Certificate Authentication System
"""
```

```
try:
    send_mail(
        subject,
        message,
        settings.DEFAULT_FROM_EMAIL,
        [email],
        fail_silently=False,
    )
    return True, "OTP sent successfully"
except Exception as e:
    logger.error(f"Error sending OTP email: {str(e)}")
    return False, f"Failed to send OTP: {str(e)}"
```